

Documentation Read_For_Me 1.0

Read_For_Me - Documentation API¶

Cette documentation est générée automatiquement à partir des docstrings du code Python du projet Read_For_Me.

Elle décrit les modules principaux de l'application :

- `main`
- `config`
- `core`
- `hardware`
- `modes`

Read_For_Me¶

config package¶

Submodules¶

config.config_loader module¶

Charge et valide la configuration centrale depuis `config/config.toml`.

Le loader renvoie un dictionnaire unique pour toute l'application avec valeurs par défaut fusionnées puis validées.

config.config_loader.load_config() → Dict[str, Any]¶

Charge `config.toml`, fusionne les valeurs par défaut puis valide le résultat.

config.config_loader.get_app_config() → Dict[str, Any]¶

Retourne la configuration complète validée de l'application.

config.config_loader.get_speaker_config() → Dict[str, Any]¶

Retourne uniquement la section `speaker` de la configuration validée.

Module contents¶

core package¶

Submodules¶

core.config_loader module¶

Pont de compatibilité vers le loader central de configuration.

Le point d'entrée officiel est `config.config_loader`.

`core.config_loader.get_config()` → `Dict[str, Any]`¶

Alias lisible pour récupérer la configuration complète.

core.mode_base module¶

`core.mode_base`¶

Définit la classe abstraite `Mode`, base commune à **tous les modes fonctionnels** de l'assistant (date/heure, multimètre, mode test, etc.).

Rôle de cette classe¶

Un *mode* représente un « contexte » d'utilisation de l'assistant : par exemple « Mode Date et heure », « Mode Multimètre », etc. À un instant donné, **un seul mode est actif**. Le changement de mode est piloté par le sélecteur rotatif et orchestré par `ModeManager`.

Cette classe définit le **contrat minimal** qu'un mode doit respecter :

- un **nom** lisible (name) pour l'annonce vocale du mode,
- **des hooks de cycle de vie** :
 - `on_enter()` : appelé quand on arrive sur le mode,
 - `on_exit()` : appelé quand on quitte le mode,
- **des callbacks liés au bouton poussoir** :
 - `on_short_press()` : appui court,
 - `on_long_press()` : appui long,
 - `on_double_press()` : double appui rapide (optionnel, valeur par défaut : ne fait rien).

Cette classe ne connaît rien : - ni de la synthèse vocale (Speaker), - ni du matériel (rotary, BLE, etc.), - ni du ModeManager qui l'utilise.

Elle sert uniquement de **base commune**, pour garder une interface cohérente entre tous les modes.

class core.mode_base.Mode(name: str)

Bases : ABC

Classe de base abstraite pour tous les modes de l'assistant.

Chaque mode concret (ex: ModeDateHeure, ModeMultimetre, etc.) doit **hériter** de cette classe et implémenter au minimum :

- on_short_press() : réaction à un appui court sur le bouton,
- on_long_press() : réaction à un appui long sur le bouton.

Les hooks suivants sont **optionnels** (no-op par défaut) :

- on_enter() / on_exit() : lifecycle du mode (drivers, timers...).
- on_double_press() : double appui sur le bouton rotatif.
- on_key_pressed(key) : touche clavier reçue du ModeManager.
- on_keypad_key(key) : alias de compatibilité de on_key_pressed. Seules les touches NON-globales sont transmises ici — les touches réservées au volume et à la vitesse sont interceptées par le ModeManager avant d'arriver dans les modes.

__init__(name: str)

Initialise un mode abstrait.

Paramètres:

name – Nom lisible du mode, utilisé pour les annonces vocales (exemples : « Machine à lire », « Multimètre », « Date et heure »).

on_enter() → None

Hook appelé quand on **arrive** sur ce mode.

Ce hook est invoqué par le ModeManager lorsque le sélecteur rotatif pointe sur ce mode et que ce mode devient le mode actif.

Par défaut, cette méthode ne fait rien. Les sous-classes peuvent la surcharger pour, par exemple :

- démarrer une tâche ou un driver matériel,
- initialiser un état interne,

- annoncer un message spécifique au mode, etc.

on_exit() → None¶

Hook appelé quand on **quitte** ce mode.

Ce hook est invoqué par le ModeManager juste avant de basculer vers un autre mode (lors d'une rotation stable du sélecteur).

Par défaut, cette méthode ne fait rien. Les sous-classes peuvent la surcharger pour, par exemple :

- arrêter proprement un driver,
- libérer des ressources,
- annuler des timers, etc.

abstractmethod on_short_press() → None¶

Appui **court** sur le bouton poussoir lorsque ce mode est actif.

Cette méthode **doit** être implémentée par chaque mode concret.

Exemple d'usage typique :

- dans un mode « Multimètre » : annoncer la dernière mesure,
- dans un mode « Date et heure » : lire l'heure actuelle,
- dans un mode « Lecture » : lire la ligne ou l'élément suivant.

abstractmethod on_long_press() → None¶

Appui **long** sur le bouton poussoir lorsque ce mode est actif.

Cette méthode **doit** être implémentée par chaque mode concret.

Exemple d'usage typique :

- afficher ou annoncer un état détaillé,
- activer une fonction secondaire,
- basculer dans un sous-mode, etc.

on_double_press() → None¶

Double appui rapide sur le bouton poussoir lorsque ce mode est actif.

Ce hook est **optionnel** : par défaut, il ne fait rien. Il peut être surchargé par un mode qui souhaite proposer un comportement spécifique sur double clic.

Exemple d'usage typique :

- dans le mode « Multimètre » : activer / désactiver la lecture automatique toutes les X secondes,
- dans un autre mode : activer un mode « favori » ou une fonction avancée sans surcharger le simple appui court.

on_key_pressed(key: str) → None¶

Point d'entrée recommandé pour les touches keypad non-globales.

Par défaut, délègue vers on_keypad_key pour préserver la compatibilité des modes existants.

on_keypad_key(key: str) → None¶

Alias de compatibilité pour la gestion des touches keypad.

Les nouveaux modes devraient préférer surcharger on_key_pressed. Les modes existants qui surchargent on_keypad_key restent entièrement compatibles.

Règle importante¶

Seules les touches NON réservées par le ModeManager sont transmises ici. Les touches globales (volume+, volume-, vitesse+, vitesse-) sont interceptées en amont et ne parviennent jamais dans cette méthode.

Paramètres¶

key :

Chaîne représentant la touche pressée (« 1 »-« 9 », « 0 », « A »-« D », « * », « # »).

Exemple d'usage¶

Dans ModeReadingMachine :

```
def on_keypad_key(self, key: str) -> None:
```

```
if key == « 1 »:
```

```
self._start_capture_pipeline()
```

```
elif key == « 5 »:
```

```
self._toggle_pause()
```

```
...
```

core.mode_manager module¶

core.mode_manager¶

Orchestrateur central des modes de l'assistant.

Responsabilités¶

1. Gérer la liste des modes et le mode courant.

2. Changer de mode quand le sélecteur rotatif tourne (set_index).
3. Router les événements bouton rotatif vers le mode actif (handle_short_press, handle_long_press, handle_double_press).
4. Optionnellement gérer un clavier matriciel 4x4 (Keypad4x4) : - Les touches « globales » (volume +/-, vitesse +/-) sont interceptées ici et traitées directement via le Speaker. Elles sont actives dans TOUS les modes, tout le temps.
 - Les autres touches sont transmises au mode actif via Mode.on_keypad_key(key). Seuls les modes qui le souhaitent surchargent cette méthode (sinon : no-op).

Architecture du clavier¶

Le Keypad4x4 est démarré UNE SEULE FOIS dans __init__ et tourne en continu, contrairement à l'ancienne approche où chaque mode le démarrait/arrêtait dans on_enter/on_exit.

Cela permet aux touches globales d'être disponibles dans n'importe quel mode, sans aucune gestion spécifique dans les modes eux-mêmes.

Touches globales (non-surchargeables par les modes)¶

```
KEY_GLOBAL_VOL_UP = « 8 » → volume + VOLUME_STEP
KEY_GLOBAL_VOL_DOWN = « 2 » → volume - VOLUME_STEP
KEY_GLOBAL_SPEED_UP = « 9 » → vitesse × SPEED_FACTOR_UP
KEY_GLOBAL_SPEED_DOWN = « 7 » → vitesse × SPEED_FACTOR_DOWN
```

Un bip de feedback est joué à chaque changement (aigu=+, grave=-).

```
class core.mode_manager.ModeManager(modes: List[Mode], speaker: Speaker,  
keypad=None, key_global_vol_up: str = '8', key_global_vol_down: str = '2',  
key_global_speed_up: str = '9', key_global_speed_down: str = '7', volume_step: int = 4,  
speed_factor_up: float = 1.25, speed_factor_down: float = 0.8)¶
```

Bases : **object**

Chef d'orchestre des modes de l'assistant.

Paramètres¶

modes :

Liste ordonnée de modes (index 0 = position 0 du rotatif).

speaker :

Speaker central — utilisé pour les annonces et les réglages globaux (volume, vitesse, bip).

keypad :

Instance de Keypad4x4 (optionnel). Si fourni, le ModeManager démarre le clavier et gère les touches globales + la délégation vers le mode courant. Si None, le clavier est ignoré.

volume_step :

Nombre de points ALSA ajoutés/retirés par pression (0–100).

speed_factor_up :

Multiplicateur appliqué à la vitesse pour une augmentation.

speed_factor_down :

Multiplicateur appliqué à la vitesse pour une diminution.

KEY_GLOBAL_VOL_UP = '8'¶

KEY_GLOBAL_VOL_DOWN = '2'¶

KEY_GLOBAL_SPEED_UP = '9'¶

KEY_GLOBAL_SPEED_DOWN = '7'¶

GLOBAL_KEYS = frozenset({'2', '7', '8', '9'})¶

VOLUME_STEP: int = 4¶

SPEED_FACTOR_UP: float = 1.25¶

SPEED_FACTOR_DOWN: float = 0.8¶

__init__(modes: List[Mode], speaker: Speaker, keypad=None, key_global_vol_up: str = '8', key_global_vol_down: str = '2', key_global_speed_up: str = '9', key_global_speed_down: str = '7', volume_step: int = 4, speed_factor_up: float = 1.25, speed_factor_down: float = 0.8) → None¶

Initialise le ModeManager.

Démarre avec le premier mode (index 0) : - annonce vocale du mode, - appel de on_enter() sur ce mode.

Si un keypad est fourni, il est démarré immédiatement et tourne en continu pour toute la durée de vie de l'assistant.

property current_index: int¶

Index du mode courant (0-based).

property current_mode: Mode¶

Instance du mode courant.

handle_key_pressed(key: str) → None¶

Point d'entrée unique pour toutes les touches du clavier.

Traitement¶

1. Si la touche est « globale » → action immédiate (volume/vitesse) + bip de feedback.
Ne parvient JAMAIS au mode courant.

2. Sinon → délégation à `self._current_mode.on_keypad_key(key)`. Si le mode ne surcharge pas cette méthode, c'est un no-op.

set_index(new_index: int, direction: str | None = None) → None

Change le mode actif.

Appelé par RotarySelector lorsqu'une rotation stable est détectée.

Paramètres

new_index :

Nouvel index ($0 \leq \text{index} < \text{len}(\text{modes})$).

direction :

Sens de rotation (« sens horaire » / « sens inverse ») — utilisé pour les logs. Pas d'effet fonctionnel pour l'instant.

Comportement

1. Coupe tout audio en cours (`stop_all_audio`).
2. `on_exit()` sur l'ancien mode.
3. Annonce le nouveau mode (nom mis en queue en premier).
4. `on_enter()` sur le nouveau mode.

handle_short_press() → None

Appui court sur le bouton rotatif.

Délégué au mode courant via `on_short_press()`.

handle_long_press() → None

Appui long sur le bouton rotatif.

Délégué au mode courant via `on_long_press()`.

handle_double_press() → None

Double appui rapide sur le bouton rotatif.

Délégué au mode courant via `on_double_press()`.

core.mode_registry module

core.mode_registry

Ce module définit le **registre des modes** disponibles dans l'application.

Pourquoi un registre ?

On veut éviter de modifier `main.py` à chaque ajout de mode.

Avec un registre : - config/settings.toml décide quels modes sont actifs et dans quel ordre.
- main.py se contente de :

- lire la config,
- instancier les modes via ce registre.

Cela rend l'architecture : - **extensible** (ajouter un mode = 1 import + 1 ligne dans le registre),
- **configurable** (activer/désactiver/re-ordonner via TOML), - **propre** (pas de if/else ou de liste codée en dur dans main).

Contrat¶

- Les clés (ex: « datetime ») doivent correspondre aux valeurs déclarées dans [modes].enabled du fichier config/config.toml.
- Chaque valeur du registre est une **classe** dérivée de Mode (et non une instance).

Exemple¶

Dans config/config.toml :

```
[modes] enabled = [« datetime », « dummy », « multimeter »]
```

Alors main.pyinstanciera dans cet ordre : ModeDateHeure, ModeDummy, ModeMultimetre.

core.speaker module¶

core.speaker¶

Sortie audio centrale de l'assistant — deux canaux indépendants :

1. CANAL ANNONCES (say / speak) ————— - File d'attente (Queue) pour ordonner les annonces. - Chaque annonce = WAV Piper joué par un sous-processus mplayer

BLOQUANT (non-slave). Le thread queue attend la fin réelle du fichier avant de passer au suivant.

- Interruptible instantanément via stop_all_audio() → terminate() sur le sous-processus en cours.

2. CANAL LECTURE LONGUE (play / PlaybackHandle)

————— - mplayer en mode slave (-slave -idle) pour contrôle fin :

pause, reprise, seek, vitesse.

- Utilisé exclusivement par ModeReadingMachine.
- Indépendant de la queue.

VOLUME GLOBAL

- Contrôlé via amixer (ALSA) → affecte tout (annonces + lecture).
- `set_volume_global()` est appellable depuis n'importe quel mode.

ISOLATION DES MODES

- `stop_all_audio()` : vide la queue + tue le sous-processus d'annonce en cours + envoie stop au player long-form.
- Appelé par ModeManager avant chaque changement de mode.

class core.speaker.PlaybackHandle(*player: _AudioPlayer*)¶

Bases : `object`

Contrôle d'une lecture longue (machine à lire).

Méthodes disponibles : pause, resume, stop, seek, set_speed.

pause() → None¶

resume() → None¶

stop() → None¶

seek(*seconds: int*) → None¶

set_speed(*speed: float*) → None¶

class core.speaker.Speaker¶

Bases : `object`

Synthèse vocale centrale de l'assistant.

Interface publique¶

`say(text, blocking=False)` → annonce courte (queue) `speak(text)` → alias `say()` non-bloquant
`play(path)` → `PlaybackHandle` → lecture longue (machine à lire) `play_sound(path, blocking)` → son court hors-queue (`aplay`) `stop_all_audio()` → coupe tout (changement de mode)
`set_volume_global(percent)` → volume ALSA global `set_global_speed(speed)` → vitesse globale (0.5 – 2.0) `close()` → arrêt propre

synthesize_to_file(*text: str, output_path: str*) → None¶

Synthétise “text” via Piper et écrit le WAV à “output_path”.

Méthode publique utilisée par les modules extérieurs (ex: pipeline machine à lire) qui veulent utiliser la voix Piper du Speaker sans charger un second modèle en mémoire.

Paramètres¶

text :

Texte à synthétiser.

output_path :

Chemin absolu du fichier WAV de sortie. Le dossier parent doit exister.

raises ValueError:

Si le texte est vide.

play_notification(blocking: bool = False) → bool¶

Joue le son de notification de volume/vitesse.

Renvoie:

True si un fichier de notification a été trouvé et tenté, False sinon.

Type renvoyé:

bool

beep(direction: str = 'up') → None¶

Alias de compatibilité : joue le son de notification.

say(text: str, blocking: bool = False) → None¶

Annonce courte — synthétisée par Piper, lue via la queue.

*Paramètres¶***text :**

Texte à synthétiser.

blocking :

Si True, attend que CETTE annonce (et toutes celles déjà en queue avant elle) soit terminée avant de retourner. Utiliser pour s'assurer qu'une annonce est entendue avant de poursuivre l'initialisation.

speak(text: str) → None¶

Alias non-bloquant de say() — compatibilité avec l'ancien code.

play(path: str) → PlaybackHandle¶

Lecture longue (machine à lire).

Coupe les annonces en cours et lance la lecture via mplayer slave. Retourne un PlaybackHandle pour contrôle fin.

*Paramètres¶***path :**

Chemin vers le fichier WAV à lire.

play_sound(path: str, blocking: bool = False) → None¶

Son court hors-queue (effets sonores : shutter, erreur, etc.).

Utilise aplay (ALSA, très léger) — aucune interférence avec la queue ni le player long-form.

stop_all_audio() → None¶

Coupe tout l'audio immédiatement.

Actions¶

1. Vide la queue (et supprime les WAVs en attente).
2. Tue le sous-processus d'annonce en cours (terminate).
3. Arrête le player long-form (mplayer slave → stop).

Appelé par ModeManager à chaque changement de mode.

set_volume_global(percent: int, mixer: str | None = None) → None¶

Règle le volume ALSA global (0 – 100).

Affecte toutes les sorties audio (annonces + lecture longue). Le mixer est détecté automatiquement si non précisé. La valeur est mémorisée dans `_current_volume` pour permettre aux modes de lire le niveau courant via `get_volume()`.

get_volume() → int¶

Retourne le volume global courant (0 – 100).

Valeur mémorisée à chaque appel de `set_volume_global()`. Utilisée par les modes pour effectuer des incréments relatifs (ex : volume +4, volume -4) sans interroger ALSA.

get_speed() → float¶

Retourne la vitesse de lecture globale courante (0.5 – 2.0).

Valeur mémorisée à chaque appel de `set_global_speed()`. Utilisée par les modes pour effectuer des incréments relatifs.

set_global_speed(speed: float) → None¶

Règle la vitesse globale (0.5 – 2.0).

S'applique : - Aux prochaines annonces (utilisé au moment du subprocess). - Au player long-form immédiatement (commande mplayer).

close() → None¶

Arrêt propre du Speaker.

Interrompt les lectures en cours, vide la queue, supprime les fichiers temporaires.

Module contents¶

hardware package¶

Subpackages¶

[hardware.devices package¶](#)

[Submodules¶](#)

[hardware.devices.caliper module¶](#)

Driver pour le pied à coulisse numérique (version NRF24).

[Vue d'ensemble¶](#)

Ce module assure le lien entre le transport radio brut et la logique métier :

1.
 - Structure les données décodées : valeur, unité (mm/pouce) et signe.
2.
 - Initialise le transport Nrf24Transport.
 - Décode la structure binaire MessageStruct envoyée par l'ATtiny85.
 - Gère les notifications d'état (prêt/erreur) vers le mode actif.

[Protocole Binaire \(Caliper_TX_24b_ATtiny85\)¶](#)

La trame reçue via NRF24 comporte au minimum 6 octets (Little Endian) : - Octets 0 à 3 : float (4 octets) -> Valeur numérique brute. - Octet 4 : bool (1 octet) -> Unité (0 = mm, 1 = pouce). - Octet 5 : bool (1 octet) -> Signe (0 = plus, 1 = moins).

class hardware.devices.caliper.CaliperData(value: float, unit_name: str, is_negative: bool)¶

Bases : [object](#)

Représente une mesure décodée issue du pied à coulisse.

[Attributs¶](#)

valuefloat

Valeur numérique de la mesure (ex: 12.45).

unit_namestr

Nom de l'unité (« millimètre » ou « pouce »).

is_negativebool

True si la mesure est négative, False sinon.

value: float

unit_name: str

is_negative: bool

class hardware.devices.caliper.CaliperDriver(*on_status: Callable[[str], None],
on_new_measure: Callable[[CaliperData], None]*)

Bases : **object**

Driver haut niveau pour le pied à coulisse sans fil.

Ce driver utilise le transport NRF24 pour recevoir des paquets et les transformer en objets CaliperData utilisables par l'assistant vocal.

__init__(*on_status: Callable[[str], None], on_new_measure: Callable[[CaliperData], None]*)

Initialise le driver du pied à coulisse.

Paramètres

on_statusCallable[[str], None]

Callback pour signaler l'état du matériel (ex: « Pied à coulisse prêt »).

on_new_measureCallable[[CaliperData], None]

Callback appelé automatiquement à chaque nouvelle mesure reçue.

start() → None

Démarre le transport radio et prépare la réception.

Cette méthode tente d'initialiser le module NRF24. Le résultat est communiqué via le callback on_status.

stop() → None

Arrête proprement le transport et libère les ressources radio.

get_last_measure() → CaliperData | None

Retourne la toute dernière mesure mémorisée.

Renvoie:

La mesure si elle existe, sinon None.

Type renvoyé:

CaliperData or None

hardware.devices.esp32_thermometer module

hardware.devices.esp32_thermometer

Driver pour le thermomètre connecté ESP32.

Vue d'ensemble¶

Ce module fournit : 1. ThermometerData : structure de donnée propre (valeur float). 2. ThermometerDriver : pilote haut niveau utilisant WifiClient.

Il est conçu pour être utilisé par ModeThermometre.

class hardware.devices.esp32_thermometer.ThermometerData(value: float, unit: str = 'Celsius', timestamp: float = 0.0)¶

Bases : **object**

Représente une mesure de température.

value: float¶

unit: str = 'Celsius'¶

timestamp: float = 0.0¶

classmethod from_raw(raw_str: str) → ThermometerData¶

Décode la chaîne brute "22.5".

class hardware.devices.esp32_thermometer.ThermometerDriver(*, target_url: str = 'http://thermo.local/api', on_status: Callable[[str], None] | None = None, on_new_measure: Callable[[ThermometerData], None] | None = None)¶

Bases : **object**

Driver haut niveau pour le thermomètre WiFi.

Responsabilités : - Piloter le transport WiFi. - Décoder les données brutes. - Fournir des messages de statut en français pour le Speaker.

start() → None¶

Démarre la surveillance.

stop() → None¶

Arrête la surveillance.

property is_connected: bool¶

Indique si le capteur est joignable.

get_last_measure() → Dict[str, float | None]¶

Retourne la dernière mesure sous forme de dict simple. Format compatible pour une consommation facile par le Mode.

hardware.devices.owon_multimeter module¶

Driver pour le multimètre OWON 16.

Vue d'ensemble ¶

Ce module fournit deux briques :

1.

- Représente **une trame de 6 octets** envoyée par le multimètre OWON 16.
- Décode la structure binaire (bits décimale / overflow / fonction / flags / valeur / signe).
- **Présente les informations dans un objet Python lisible :**
 - `value` : valeur numérique en float (avec signe et décimale appliqués),
 - `unit_name` : nom de la fonction, ex. « MILLI VOLT DC »,
 - `flags` booléens : `overflow`, `auto_ranging`, `data_hold_mode`, `relative_mode`, `low_battery`,
 - `raw_data` : la trame brute (6 octets) pour debug / log.

2.

- Driver **haut niveau** pour le multimètre OWON basé sur le transport générique BleClient.
- **Responsabilités :**
 - gérer la recherche / connexion BLE (`start()`, `stop()`, `is_connected`),
 - décoder les trames brutes en `OwonMultimeterData`,
 - mémoriser la **dernière mesure** (`get_last_measure()`),
 - annoncer intelligemment les **changements de mode de mesure** (fonction du multimètre) via un callback `on_status(text)` (par ex. le `Speaker`).
- Il ne fait **aucune synthèse vocale directement** : il envoie du texte vers le `Mode`, qui lui-même appelle le `Speaker`.

Couche & responsabilités ¶

- **Ce module connaît :**
 - le protocole binaire OWON (layout des 6 octets),
 - les codes de fonction (`OWON_FUNCTION`),
 - la logique d'annonce des changements de mode (hystérésis).
- **Il ne connaît PAS :**
 - le sélecteur rotatif (géré ailleurs),

- l'architecture globale des modes,
- la façon dont le texte sera lu (voix, langue, etc.).

hardware.devices.owon_multimeter.OWON_FUNCTION: Dict[str, int] = {'AMPERE_AC': 7708, 'AMPERE_DC': 7700, 'CELSIUS': 7748, 'CONTINUITY_TEST': 7772, 'DIODE_TEST': 7764, 'FAHRENHEIT': 7756, 'FARAD': 7724, 'HERTZ': 7732, 'KILO_OHM': 7717, 'MEGA_OHM': 7718, 'MICRO_AMPERE_AC': 7706, 'MICRO_AMPERE_DC': 7698, 'MICRO_FARAD': 7722, 'MILLI_AMPERE_AC': 7707, 'MILLI_AMPERE_DC': 7699, 'MILLI_FARAD': 7723, 'MILLI_VOLT_AC': 7691, 'MILLI_VOLT_DC': 7683, 'NANO_FARAD': 7721, 'NEAR_FIELD': 7788, 'OHM_NORMAL': 7716, 'PERCENTAGE': 7740, 'VOLT_AC': 7692, 'VOLT_DC': 7684}

Dictionnaire de mapping « nom symbolique » -> code 13 bits « function_selector »

Ces valeurs proviennent de la documentation / reverse-engineering du multimètre OWON 16. Le champ « fonction » est encodé sur 13 bits dans les deux premiers octets de chaque trame.

Remarque importante sur les milliampères

Dans la doc d'origine, les codes **AC/DC pour les milliampères** sont inversés par rapport à ce que le multimètre envoie réellement. Après tests sur le matériel, nous avons corrigé le mapping :

- 7699 -> « MILLI_AMPERE_DC »
- 7707 -> « MILLI_AMPERE_AC »

Cela garantit que les annonces vocales et unit_name sont cohérents avec ce qui est affiché sur l'écran du multimètre (évite de dire « AC » quand l'écran est en « DC » et inversement).

hardware.devices.owon_multimeter.OWON_MODE_LABEL_FR: Dict[int, str] = {7683: 'millivoltmètre courant continu', 7684: 'voltmètre courant continu', 7691: 'millivoltmètre courant alternatif', 7692: 'voltmètre courant alternatif', 7698: 'microampèremètre courant continu', 7699: 'milliampèremètre courant continu', 7700: 'ampèremètre courant continu', 7706: 'microampèremètre courant alternatif', 7707: 'milliampèremètre courant alternatif', 7708: 'ampèremètre courant alternatif', 7716: 'ohmmètre', 7717: 'kilo ohmmètre', 7718: 'méga ohmmètre', 7721: 'mesure de capacité en nanofarad', 7722: 'mesure de capacité en microfarad', 7723: 'mesure de capacité en millifarad', 7724: 'mesure de capacité en farad', 7732: 'fréquencemètre', 7740: 'mesure de pourcentage', 7748: 'thermomètre en degrés Celsius', 7756: 'thermomètre en degrés Fahrenheit', 7764: 'test de diode', 7772: 'test de continuité', 7788: 'détection de champ électrique'}

Mapping « code fonction OWON » -> phrase FR annoncée

Utilisé par OwonMultimeterDriver pour annoncer les changements de mode (ex : « Mode voltmètre courant continu. »). Ces libellés sont pensés pour être prononcés directement par la synthèse vocale.

```
class hardware.devices.owon_multimeter.OwonMultimeterData(raw_data: List[int],  
decimal_places: int, overflow: int, unit: int, unit_name: str, data_hold_mode: int,  
relative_mode: int, auto_ranging: int, low_battery: int, value: float, sign: int)
```

Bases : **object**

Représente une trame décodée du multimètre OWON 16 (6 octets).

Structure binaire (vue « fabricant »)

Les 6 octets sont organisés comme suit (Little Endian) :

```
decimal_places 0 2 overflow 2 1 function_selector 3 13 data_hold_mode 16 1  
relative_mode 17 1 auto_ranging 18 1 low_battery 19 1 not_used_1 20 4  
not_used_2 24 8 value 32 15 sign 47 1
```

Interprétation

- decimal_places : position de la décimale (0, 1, 2 ou 3).
- overflow : dépassement de gamme (0 = OK, 1 = overflow).
- **function_selector (ici unit) :**
code 13 bits mappé via OWON_FUNCTION vers un nom symbolique.
- data_hold_mode : flag HOLD (gel d’affichage).
- relative_mode : mode relatif (REL).
- auto_ranging : autorange actif ou non.
- low_battery : batterie faible.
- value : valeur brute signée (15 bits + signe), mise à l’échelle.
- sign : 0 = positif, 1 = négatif.

Attributs principaux exposés

raw_dataList[int]

Trame brute reçue (6 octets).

decimal_placesint

Nombre de décimales à appliquer à la valeur.

overflowint

Flag overflow (0 ou 1).

unitint

Code numérique de la fonction (13 bits).

unit_namestr

Nom lisible de la fonction (ex: « MILLI VOLT DC », « OHM NORMAL », ...).

data_hold_modeint

Flag HOLD (0 ou 1).

relative_modeint

Flag REL (0 ou 1).

auto_rangingint

Flag AutoRange (0 ou 1).

low_batteryint

Flag batterie faible (0 ou 1).

valuefloat

Valeur numérique finale (signe et décimale appliqués).

signint

Flag de signe (0 = positif, 1 = négatif).

raw_data: List[int]

decimal_places: int

overflow: int

unit: int

unit_name: str

data_hold_mode: int

relative_mode: int

auto_ranging: int

low_battery: int

value: float

sign: int

classmethod from_raw(raw_data: List[int]) → **OwonMultimeterData**

Construit une instance à partir d'une trame brute (6 octets).

Paramètres

raw_dataList[int]

Liste de 6 octets (0–255) reçus depuis le multimètre.

returns:

Instance décodée, prête à être utilisée (value, flags, etc.).

rtype:

OwonMultimeterData

raises ValueError:

Si la longueur de raw_data n'est pas égale à 6.

```
class hardware.devices.owon_multimeter.OwonMultimeterDriver(*, on_status:
Callable[[str], None] | None = None, on_new_measure:
Callable[[OwonMultimeterData], None] | None = None, name_filter: str = 'BDM',
address_filter: str | None = None, characteristic_uuid: str = '0000fff4-0000-1000-8000-
00805f9b34fb', scan_timeout: float = 4.0, retry_delay: float = 15.0, reconnect_delay:
float = 2.0, listen_poll_interval_sec: float = 0.5, mode_announce_delay: float = 0.7)¶
Bases : object
```

Driver haut niveau pour le multimètre OWON 16.

Rôle¶

- Utiliser BleClient comme **transport BLE générique**.
- Décoder les trames brutes reçues via BLE en objets OwonMultimeterData.
- Mémoriser la dernière mesure décodée (_last_data).
- Annoncer les **changements de mode de mesure** (tension, courant, ohms, ...) via une logique d'hystérésis (temps minimum de stabilité).
- **Exposer une API simple pour les Modes :**
 - start() / stop() pour lancer/arrêter la surveillance BLE.
 - is_connected pour vérifier l'état de connexion.
 - get_last_measure() pour récupérer la dernière mesure sous forme de dict.

Couche d'abstraction¶

- Ce driver ne fait pas de synthèse vocale. Il pousse simplement des **messages texte** dans le callback on_status.
- Le Mode (ex: ModeMultimetre) se charge ensuite de passer ces messages au Speaker central.

```
__init__(*, on_status: Callable[[str], None] | None = None, on_new_measure:
Callable[[OwonMultimeterData], None] | None = None, name_filter: str = 'BDM',
address_filter: str | None = None, characteristic_uuid: str = '0000fff4-0000-1000-8000-
00805f9b34fb', scan_timeout: float = 4.0, retry_delay: float = 15.0, reconnect_delay:
float = 2.0, listen_poll_interval_sec: float = 0.5, mode_announce_delay: float = 0.7) →
None¶
```

Initialise le driver OWON.

Paramètres¶

on_statusOptional[Callable[[str], None]]

Callback pour annoncer des messages utilisateur (via Speaker). Exemple de messages :

- « Multimètre connecté. Vous pouvez effectuer vos mesures. »

- « Mode voltmètre courant continu. »
- « Multimètre non trouvé. Assurez-vous qu'il est allumé... »

on_new_measureOptional[Callable[[OwonMultimeterData], None]]

Callback appelé à chaque **nouvelle trame décodée**. Permet au Mode d'enregistrer directement la dernière mesure brute.

retry_delayfloat

Délai (en secondes) entre deux tentatives lorsqu'aucun multimètre n'est trouvé lors du scan BLE.

start() → None

Démarre la recherche / connexion au multimètre OWON.

Effets

- Lance le BleClient dans un thread séparé.
- Le driver se met à l'écoute des trames et des événements d'état.

stop() → None

Arrête la surveillance du multimètre.

Effets

- Demande l'arrêt du BleClient (stop de la boucle de monitoring).
- À terme, la connexion BLE est fermée et l'état repasse à « stopped ».

property is_connected: bool

Indique si le multimètre est actuellement connecté au niveau BLE.

Renvoie:

True si BleClient est connecté au périphérique OWON, False sinon.

Type renvoyé:

bool

get_last_measure() → Dict[str, object | None]

Retourne la dernière mesure sous forme de dict simple, prêt à consommer.

Ce format est volontairement **générique** pour être facile à utiliser côté Modes / IHM / logs, sans exposer directement l'objet OwonMultimeterData.

Clés du dictionnaire

- « value » : float ou None
- « unit_name » : str ou None (ex: « MILLI VOLT DC », « OHM NORMAL »)
- « overflow » : bool ou None

- « auto_ranging » : bool ou None
- « data_hold_mode »: bool ou None
- « relative_mode » : bool ou None

returns:

Dictionnaire avec les champs ci-dessus. Si aucune mesure n'a encore été reçue, toutes les valeurs sont à None.

rtype:

dict

hardware.devices.text_reader_device module ¶

hardware.devices.text_reader_device ¶

Device « machine à lire » (couche au-dessus du hardware brut).

Responsabilité ¶

Orchestrer les trois premières étapes de la chaîne de lecture :

1. Capture caméra → <basename>.jpg (via PiCamera)
2. OCR → <basename>_raw.txt (via pytesseract)
3. Nettoyage du texte → <basename>.txt (logique propre)

Ce module NE gère PAS :

- La synthèse vocale (TTS) : déléguée au Speaker central via Speaker.synthesize_to_file(), ce qui évite de charger le modèle Piper une deuxième fois en mémoire.
- Le keypad / les touches : gérés par le mode (ModeReadingMachine).
- Le playback / pause / seek : gérés par Speaker/mplayer côté core.
- La logique de mode (on_enter/on_exit, mapping touches, etc.).

Fichiers produits ¶

Pour un basename donné (ex« /tmp/readforme/scan ») :

- Image brute : /tmp/readforme/scan.jpg
- Texte OCR : /tmp/readforme/scan_raw.txt
- Texte propre : /tmp/readforme/scan.txt

Le fichier WAV est produit EN DEHORS de ce module, par le mode qui appelle Speaker.synthesize_to_file() puis Speaker.play().

Pourquoi cette séparation ?

L'ancien code embarquait le TTS (pico2wave, puis Piper) directement ici. Cela impliquait de charger le modèle Piper une seconde fois, consommant inutilement de la RAM sur le Raspberry Pi. Désormais, un seul modèle Piper est chargé (dans `Speaker.__init__`) et partagé via `Speaker.synthesize_to_file()`.

```
class hardware.devices.text_reader_device.OCRConfig(lang: str = 'fra', psm: int = 6,  
oem: int = 1, preprocess_enabled: bool = True, autocontrast_cutoff: int = 2,  
sharpen_factor: float = 1.2, threshold: int = 0, timeout_sec: float = 20.0, fallback_psm:  
tuple[int, ...] = (7, 11, 3), save_debug_ocr_input: bool = True)
```

Bases : `object`

Paramètres de la reconnaissance optique de caractères (OCR).

Attributs

lang :

Code langue tesseract (ex : « fra » pour le français, « eng » pour l'anglais). Doit correspondre à un pack de langues installé.

psm :

Page Segmentation Mode tesseract (–psm). 3 = détection automatique (valeur recommandée pour documents). Voir : `tesseract –help-psm`

```
lang: str = 'fra'
```

```
psm: int = 6
```

```
oem: int = 1
```

```
preprocess_enabled: bool = True
```

```
autocontrast_cutoff: int = 2
```

```
sharpen_factor: float = 1.2
```

```
threshold: int = 0
```

```
timeout_sec: float = 20.0
```

```
fallback_psm: tuple[int, ...] = (7, 11, 3)
```

```
save_debug_ocr_input: bool = True
```

```
class hardware.devices.text_reader_device.TextReaderDevice(camera: PiCamera, *,  
ocr: OCRConfig | None = None)
```

Bases : `object`

Device « machine à lire » — pipeline capture → OCR → texte propre.

Ce device s'arrête à la production du texte nettoyé. La synthèse vocale est gérée par le Speaker central.

API principale

`capture(basename)` → str chemin de l'image `ocr_to_text(basename)` → str chemin du texte

OCR brut `clean_text(basename)` → str chemin du texte nettoyé `run_full_pipeline(basename)`

→ str chemin du texte nettoyé (enchaîne les 3)

Convention de nommage (basename = « /tmp/readforme/scan ») :

Image brute : /tmp/readforme/scan.jpg Texte OCR : /tmp/readforme/scan_raw.txt Texte propre : /tmp/readforme/scan.txt

`__init__(camera: PiCamera, *, ocr: OCRConfig | None = None) → None`

Initialise le device.

Paramètres

camera :

Instance de PiCamera configurée (rotation, résolution...).

ocr :

Configuration OCR. Si None, utilise les valeurs par défaut (lang= »fra », psm=3).

`capture(basename: str) → str`

Capture une photo et écrit <basename>.jpg.

Paramètres

basename :

Chemin sans extension (ex : « /tmp/readforme/scan »).

Retourne

str

Chemin complet du fichier image créé.

`ocr_to_text(basename: str) → str`

Effectue l'OCR sur <basename>.jpg et écrit <basename>_raw.txt.

Utilise pytesseract avec la langue et le PSM configurés.

Paramètres

basename :

Chemin sans extension.

Retourne

str

Chemin du fichier texte brut produit.

raises FileNotFoundError:

Si l'image n'existe pas (étape capture non réalisée).

`clean_text(basename: str) → str`

Nettoie le texte OCR brut et écrit <basename>.txt.

Stratégie de nettoyage (identique à l'ancien projet) : - Les lignes non vides sont concaténées avec des espaces. - Une ligne vide introduit un saut de paragraphe (double newline). - Les traits d'union de fin de ligne (»- n ») sont supprimés

(gestion des mots coupés par l'OCR).

Paramètres¶

basename :

Chemin sans extension.

Retourne¶

str

Chemin du fichier texte nettoyé.

raises FileNotFoundError:

Si le texte brut n'existe pas (étape ocr_to_text non réalisée).

raises RuntimeError:

Si le texte nettoyé est vide (document illisible ou page vide).

run_full_pipeline(basename: str) → str¶

Exécute la chaîne complète : capture → OCR → nettoyage.

Équivalent à appeler capture(), ocr_to_text(), clean_text() dans l'ordre, avec gestion d'erreurs centralisée.

Paramètres¶

basename :

Chemin sans extension (ex : « /tmp/readforme/scan »).

Retourne¶

str

Chemin du fichier texte nettoyé (<basename>.txt). Le contenu est prêt à être synthétisé via Speaker.synthesize_to_file().

raises FileNotFoundError:

Si un fichier intermédiaire n'existe pas.

raises RuntimeError:

Si le texte résultant est vide.

Module contents¶

hardware.platform package¶

Submodules¶

hardware.platform.keypad_4x4 module

Driver clavier matriciel 4x4 (GPIO) pour Raspberry Pi.

Responsabilité

- Scanner un clavier 4x4 câblé en matrice (4 lignes / 4 colonnes)
- Déclencher un callback quand une touche est pressée
- Fournir start/stop pour activer ce périphérique uniquement dans certains modes

Choix d'API (propre pour l'archi)

- Keypad4x4.start() : démarre un thread de scan (non-bloquant)
- Keypad4x4.stop() : stoppe le thread proprement
- Keypad4x4.on_key : callback optionnel appelé quand une touche est détectée

class hardware.platform.keypad_4x4.KeypadPins(row1: 'int', row2: 'int', row3: 'int', row4: 'int', col1: 'int', col2: 'int', col3: 'int', col4: 'int')

Bases : **object**

row1: int

row2: int

row3: int

row4: int

col1: int

col2: int

col3: int

col4: int

class hardware.platform.keypad_4x4.Keypad4x4(pins: KeypadPins, *, layout: List[List[str]] | None = None, scan_interval: float = 0.02, debounce_time: float = 0.2)

Bases : **object**

Driver de scan clavier 4x4.

- Scan cyclique des lignes (row) -> lecture colonnes (col)
- Anti-rebond logiciel simple
- Détection « press » (pas maintien)

DEFAULT_LAYOUT: List[List[str]] = [['1', '2', '3', 'A'], ['4', '5', '6', 'B'], ['7', '8', '9', 'C'], ['*', '0', '#', 'D']]

on_key: Callable[[str], None] | None

start() → None

Démarre le scan clavier dans un thread.

stop() → None

Stoppe le scan clavier.

hardware.platform.pi_camera module

hardware.platform.pi_camera

Driver bas niveau pour la caméra Raspberry Pi (libcamera / rpicam).

Responsabilité

- Capturer une image via rpicam-still (Bookworm) ou libcamera-still (legacy)
- Ne contient AUCUNE logique métier (pas d'OCR, pas de TTS, pas de mode)

API

- **PiCamera.capture(output_path) → str**
Capture une photo et l'écrit dans output_path.

Notes

- Utilise subprocess.run (plus fiable que os.system).
- Lève des exceptions explicites en cas d'erreur.

```
class hardware.platform.pi_camera.CameraConfig(rotation: int = 180, timeout_ms: int = 500, width: int | None = None, height: int | None = None, command_timeout_sec: float = 10.0)
```

Bases : *object*

Configuration de capture.

rotation :

Rotation appliquée par l'outil de capture (0/90/180/270)

timeout_ms :

Délai avant capture (ms). Permet auto-exposure stable.

width / height :

Optionnels. Si None, l'outil choisit par défaut.

rotation: int = 180

timeout_ms: int = 500

width: int | None = None

height: int | None = None

command_timeout_sec: float = 10.0

```
class hardware.platform.pi_camera.PiCamera(config: CameraConfig | None = None)
```

Bases : *object*

Caméra Raspberry Pi via `rpicas-still` (recommandé) ou `libcamera-still` (legacy).

`static probe() → tuple[bool, str]`

Vérifie si une caméra est détectée et retourne (available, diagnostic).

`static is_available() → bool`

Vérifie si l'outil `libcamera/rpicam` voit au moins une caméra.

`static detect_blocking_processes() → List[str]`

Retourne best-effort la liste des processus utilisant `/dev/video*`.

`capture(output_path: str) → str`

Capture une image et l'écrit dans `output_path`.

Renvoie:

`output_path`

Type renvoyé:

`str`

Lève:

- **`RuntimeError`** – si l'outil de capture échoue
- **`FileNotFoundError`** – si le fichier n'est pas créé

`hardware.platform.rotary_selector module`

`hardware.platform.rotary_selector`

Driver bas niveau pour le **sélecteur rotatif + bouton poussoir**.

Objectif

Ce module encapsule la logique « physique » du rotateur :

- lecture des signaux **quadrature** (canaux A/B) pour la rotation,
- calcul du **sens** du geste (« sens horaire » / « sens inverse »),
- agrégation de plusieurs impulsions en un seul **geste stable**,
- protection contre les **rebonds** et gestes trop rapides,
- **gestion des appuis sur le bouton** :
 - appui court,
 - appui long,
 - double-clic (double appui court rapide).

Important : ce driver est **agnostique** de la logique métier.

- Il **ne connaît pas** les modes, ni le speaker, ni le multimètre.
- Il **expose simplement des callbacks que le reste du système branche** :
 - `on_position_changed(index, direction)`
 - `on_short_press()`
 - `on_long_press()`
 - `on_double_press()`

L'objectif est d'avoir un composant réutilisable, proprement isolé, qui s'occupe uniquement de transformer des signaux GPIO bruts en événements haut niveau exploitables par l'assistant.

```
class hardware.platform.rotary_selector.RotarySelector(pin_a: int = 17, pin_b: int = 27,  
pin_sw: int = 22, positions_count: int = 4, long_press_threshold: float = 1.0,  
gesture_settle_delay: float = 0.2, rotate_button_deadzone: float = 0.15,  
min_step_interval: float = 0.3, double_click_window: float = 0.4, button_bounce_time:  
float = 0.05)
```

Bases : `object`

Driver pour un encodeur rotatif incrémental avec bouton poussoir intégré.

Rôle

- Maintient une **position logique de mode** : 0 .. positions_count-1.
- Détecte le **sens de rotation** et produit des gestes stables.
- **Gère les appuis** :
 - court,
 - long,
 - double appui rapide (double-clic).

Stratégie UX

- Plusieurs impulsions rapprochées dans le **même sens** sont agrégées en **un seul geste** :
=> un seul changement de position / mode.
- Les allers-retours très rapides (bruit, gestes hésitants) sont filtrés via :
 - un délai minimum entre deux changements de mode (`min_step_interval`),
 - un délai de **stabilisation de geste** (`gesture_settle_delay`).

- Une **zone morte** autour des appuis bouton (`rotate_button_deadzone`) évite que des rebonds mécaniques du bouton soient interprétés comme des rotations.
- La gestion du **double-clic** repose sur une petite fenêtre de temps (`double_click_window`) : si deux appuis courts se succèdent dans ce délai, on considère qu'il s'agit d'un double appui.

Paramètres ¶

pin_aint

Broche GPIO du canal A de l'encodeur.

pin_bint

Broche GPIO du canal B de l'encodeur.

pin_swint

Broche GPIO du bouton poussoir.

positions_countint

Nombre total de positions logiques (nombre de modes disponibles).

long_press_thresholdfloat

Durée (en secondes) à partir de laquelle un appui est considéré comme « long » plutôt que « court ».

gesture_settle_delayfloat

Temps sans nouvelle impulsion avant de considérer qu'un geste de rotation est terminé et de changer effectivement de position.

rotate_button_deadzonefloat

Intervalle de temps (en secondes) autour d'un événement bouton pendant lequel on ignore les impulsions de rotation (anti-parasites).

min_step_intervalfloat

Délai minimum (en secondes) entre deux changements de position consécutifs (limite les changements trop rapides).

double_click_windowfloat

Temps maximum (en secondes) entre deux appuis courts consécutifs pour être interprétés comme un **double appui**.

Module contents ¶

hardware.transports package¶

Submodules¶

hardware.transports.ble_client module¶

hardware.transports.ble_client¶

Client BLE générique basé sur **bleak**, utilisé comme couche de transport.

Rôle du module¶

Ce module fournit un petit **service BLE réutilisable** qui :

- scanne l'environnement pour trouver un périphérique BLE cible,
- se connecte à ce périphérique,
- s'abonne à une **caractéristique** pour recevoir des notifications,
- gère les **reconnexions automatiques** en cas de perte de connexion ou d'erreur,
- expose deux callbacks indépendants de la logique métier :
 - **on_status(event: str) :**
 - informe l'appelant de l'état du client (recherche, trouvé, connecté, déconnecté, erreurs, etc.) ;
 - **on_packet(data: bytes) :**
 - transmet les **trames brutes** reçues sur la caractéristique.

Important

Ce client **ne connaît pas** la notion de « multimètre », « capteur X », etc.

- Il ne fait que gérer le **transport BLE** (scan, connexion, notifications).
- La logique métier (décodage des trames, phrases vocales, etc.) est déportée dans des drivers plus haut niveau (ex : OwonMultimeterDriver).

Il est donc possible de réutiliser cette classe pour **n'importe quel périphérique BLE**, à condition de fournir : - un filtre de nom ou d'adresse, - l'UUID de la caractéristique à écouter, - des callbacks adaptés.

```
class hardware.transports.ble_client.BleClient(*, name_filter: str | None,
characteristic_uuid: str, address_filter: str | None = None, on_status: Callable[[str],
None] | None = None, on_packet: Callable[[bytes], None] | None = None, scan_timeout:
float = 4.0, retry_delay: float = 15.0, reconnect_delay: float = 2.0,
listen_poll_interval_sec: float = 0.5)¶
```

Bases : **object**

Client BLE générique avec boucle de reconnexion automatique.

Ce composant tourne dans un **thread dédié** pour ne pas bloquer le thread principal de l'application. La logique interne utilise asyncio (scan / connexion / notifications), mais l'API publique (start(), stop(), propriétés) reste **synchrone** et simple à utiliser.

Paramètres¶

name_filterOptional[str]

Nom BLE à rechercher (exact, insensible à la casse). Exemple : « BDM » pour le multimètre OWON 16.

address_filterOptional[str]

Adresse MAC BLE à rechercher (si fournie, elle est prioritaire sur name_filter).

characteristic_uuidstr

UUID de la caractéristique sur laquelle on reçoit les notifications. Exemple : « 0000fff4-0000-1000-8000-00805f9b34fb ».

on_statusOptional[Callable[[str], None]]

Callback pour les événements d'état. Les valeurs possibles sont : - « search_start » : début d'une phase de recherche. - « not_found » : aucun device cible trouvé pour cette tentative. - « found » : device cible détecté (avant connexion). - « connecting » : tentative de connexion en cours. - « connected » : connexion établie. - « disconnected » : connexion perdue alors que le client tournait. - « error » : erreur générique (scan / boucle principale). - « connect_error » : erreur lors de la connexion ou de l'écoute. - « stopped » : la boucle principale s'est arrêtée (après stop()).

on_packetOptional[Callable[[bytes], None]]

Callback appelé à chaque paquet reçu sur la caractéristique. Le paramètre est la trame brute bytes (6 octets pour l'OWON, par ex.).

scan_timeoutfloat

Durée (en secondes) d'une phase de scan BLE (BleakScanner.discover).

retry_delayfloat

Délai (en secondes) entre deux tentatives de scan lorsqu'aucun device n'a été trouvé.

reconnect_delayfloat

Délai (en secondes) avant de retenter la connexion après une erreur.

property is_running: bool¶

Indique si la boucle de monitoring BLE est actuellement active.

Renvoie:

True si le thread BLE est démarré (start() appelé et non stoppé), False sinon.

Type renvoyé:

bool

property is_connected: bool

Indique si le client est **actuellement connecté** à un périphérique BLE.

Renvoie:

True si une connexion est active et que les notifications sont en cours, False sinon.

Type renvoyé:

bool

start() → None

Démarre la boucle de monitoring dans un **thread séparé**.

Comportement

- Si le client est déjà en cours d'exécution (`is_running == True`), la méthode ne fait rien (idempotent).
- **Sinon :**
 - crée un nouveau thread (`BleClientThread`),
 - lance `_run_asyncio_loop()` dans ce thread,
 - active la reconnexion automatique tant que `stop()` n'est pas appelé.

stop() → None

Demande l'arrêt de la boucle de monitoring BLE.

Effets

- Met `_running` à False : la boucle principale (`_main_loop`) va se terminer proprement à la prochaine itération.
- À la fin, un événement « stopped » sera émis via `on_status`, et `is_connected` passera à False.

hardware.transports.nrf24_transport module

hardware.transports.nrf24_transport

Ce module définit la classe `Nrf24Transport`, responsable de la communication bas niveau avec le module radio NRF24L01+.

Vue d'ensemble

Le transport NRF24 agit comme une couche d'abstraction au-dessus du bus SPI : - Il initialise le matériel radio (CE, CSN, Canal, Débit). - Il gère la réception des données en arrière-plan via un **thread dédié**. - Il propage les paquets bruts reçus vers les drivers de périphériques

(ex. : pied à coulisse) via un système de callback.

Objectifs du design¶

- **Robustesse** : Utilise des blocs try/except pour éviter le crash de l'application si la librairie pyrf24 est absente ou si le module radio est débranché.
- **Réactivité** : Le thread Nrf24Worker écoute en permanence le bus SPI sans bloquer la boucle principale du ModeManager.
- **Découplage** : Le transport ne connaît pas le contenu des messages ; il se contente de livrer des octets bruts (bytes).

Câblage type (Raspberry Pi 5)¶

VCC -> 3.3V | GND -> GND | CE -> GPIO 25 | CSN -> GPIO 8 (CE0) SCK -> GPIO 11 | MOSI -> GPIO 10 | MISO -> GPIO 9

```
class hardware.transports.nrf24_transport.Nrf24Transport(ce_pin: int = 25, csn_pin: int = 0, channel: int = 120, address: bytes = b'00001')¶
```

Bases : **object**

Gère la couche basse radio NRF24L01+ sur bus SPI.

Rôle¶

Cette classe encapsule la librairie pyrf24 pour fournir une interface de lecture simplifiée et threadée. Elle est conçue pour être utilisée par un driver de périphérique (comme le Pied à coulisse).

```
__init__(ce_pin: int = 25, csn_pin: int = 0, channel: int = 120, address: bytes = b'00001')¶
```

Initialise les paramètres du transport radio.

Paramètres¶

ce_pinint

Broche GPIO pour le signal Chip Enable (défaut : 25).

csn_pinint

Broche pour le signal Chip Select Not (0 pour SPI0 CE0 / GPIO 8).

channelint

Canal radio utilisé (0 à 125). Le pied à coulisse MHK utilise le 120.

addressbytes

Adresse unique du tuyau de lecture (pipe), format 5 octets.

```
start(on_packet_callback: Callable[[bytes], None]) → Tuple[bool, str]¶
```

Initialise le module radio et démarre le thread d'écoute.

Paramètres¶

on_packet_callbackCallable

Fonction appelée à chaque réception d'un nouveau paquet radio. Prend un argument de type bytes.

returns:

Un tuple contenant le succès (True/False) et un message de statut.

rtype:

(bool, str)

stop() → None¶

Arrête proprement l'écoute radio et le thread associé.

hardware.transports.wifi_client module¶

hardware.transports.wifi_client¶

Client HTTP générique pour le polling de capteurs sur réseau local. Architecture inspirée de ble_client.py.

Rôle du module¶

Ce module fournit un **service de transport HTTP réutilisable** qui :

- Tente de contacter une API HTTP cible (URL) à intervalles réguliers (polling).
- Détecte automatiquement la disponibilité du périphérique (connexion/déconnexion).
- Gère les timeouts et erreurs réseaux de manière transparente.
- **Expose deux callbacks indépendants de la logique métier :**
 - on_status(event: str) : informe de l'état (connecting, connected, error...).
 - on_data(data: str) : transmet le corps de la réponse HTTP brute.

Important

Ce client ne gère **aucun protocole spécifique** au-delà du HTTP standard. Il ne fait que transporter des chaînes de caractères reçues via HTTP.

Il est réutilisable pour tout capteur exposant une API REST simple, à condition de fournir l'URL cible.

```
class hardware.transports.wifi_client.WifiClient(*, target_url: str, on_status:  
Callable[[str], None] | None = None, on_data: Callable[[str], None] | None = None,  
poll_interval: float = 2.0, request_timeout: float = 3.0)
```

Bases : `object`

Client de Polling HTTP générique.

Ce composant tourne dans un **thread dédié** pour ne pas bloquer le thread principal.

Paramètres

target_url`str`

L'URL complète de l'API à interroger (ex: « <http://thermo.local/api> »).

on_status`Optional[Callable[[str], None]]`

Callback pour les événements d'état. Valeurs possibles : - « connecting » : tentative de contact avec l'URL (début ou reconnexion). - « connected » : périphérique joint avec succès (HTTP 200). - « disconnected » : perte de liaison (timeout ou erreur réseau). - « http_error » : le périphérique répond mais avec une erreur (404, 500...). - « stopped » : arrêt volontaire du client.

on_data`Optional[Callable[[str], None]]`

Callback appelé à chaque réponse HTTP valide. Le paramètre est le texte brut (str) de la réponse.

poll_interval`float`

Temps (en secondes) entre deux requêtes HTTP.

request_timeout`float`

Temps (en secondes) avant de considérer le périphérique comme absent.

property is_running`bool`

Indique si la boucle de polling est active.

property is_connected`bool`

Indique si la dernière communication HTTP a réussi.

start() `→ None`

Démarre la boucle de polling dans un thread séparé.

stop() `→ None`

Arrête la boucle de polling.

Module contents

Module contents¶

main module¶

modes package¶

Submodules¶

modes.mode_caliper module¶

Mode « Pied à coulisse » pour l'assistant ReadForMe.

Ce module encapsule toute la logique « métier » et l'interface utilisateur pour les mesures de distance sans fil :

- **Gestion de la voix :**
 - Toutes les annonces passent par le Speaker central.
- **Gestion du matériel :**
 - Dialogue avec le récepteur radio via CaliperDriver.
- **Logique UX (Bouton poussoir) :**
 - Appui court : Annoncer la dernière mesure reçue.
 - Appui long : Annoncer l'état de connexion radio (SPI/NRF24).
 - Double appui : Activer ou désactiver la lecture automatique périodique.

Ce mode suit le cycle de vie défini par `core.mode_base.Mode`.

class modes.mode_caliper.ModeCaliper(*speaker: Speaker*)¶

Bases : `Mode`

Mode « Pied à coulisse » de l'assistant.

Rôle¶

Fournit une interface vocale pour le pied à coulisse numérique. Il traduit les paquets radio reçus en annonces audibles et gère les interactions utilisateur via le bouton du sélecteur rotatif.

__init__(*speaker: Speaker*)¶

Initialise le mode Pied à coulisse.

Paramètres¶

speakerSpeaker

Instance du gestionnaire vocal central pour les annonces.

on_enter() → None¶

Hook appelé lors de la sélection du mode. - Désactive la lecture automatique par sécurité. - Démarre le thread de lecture radio.

on_exit() → None¶

Hook appelé quand on change de mode. - Arrête les annonces automatiques. - Coupe proprement le driver radio.

on_short_press() → None¶

Appui court : Annonce vocalement la dernière mesure mémorisée.

Gère le formatage du signe et de l'unité (millimètres ou pouces). Exemple : « moins 12.45 millimètres »

on_long_press() → None¶

Appui long : Annonce l'état du matériel. Utile pour diagnostiquer si le module NRF24 est bien branché au Pi.

on_double_press() → None¶

Double appui : Bascule l'état de la lecture automatique.

modes.mode_datetime module¶

Mode « Date et heure ».

Ce module définit un mode très simple qui exploite l'horloge système pour annoncer :

- l'heure actuelle (appui court),
- la date du jour (appui long).

Il ne gère aucune logique matérielle directement : il s'appuie sur le ModeManager (pour la sélection du mode) et sur le Speaker (pour la synthèse vocale).

class modes.mode_datetime.ModeDateHeure(speaker: *Speaker*)¶

Bases : Mode

Mode « Date et heure » de l'assistant.

Comportement UX¶

- **À l'entrée du mode (on_enter) :**
 - rien n'est annoncé ici, le ModeManager annonce déjà « Mode X : Date et heure ».

- **Appui court (on_short_press) :**
 - annonce l'heure système actuelle (ex : « Il est 14 heures 32 »).
- **Appui long (on_long_press) :**
 - annonce la date du jour (ex : « Nous sommes le mardi 5 juin 2025 »).

Remarques¶

- Les libellés de jour et de mois (%A, %B) dépendent de la locale système (ex : fr_FR recommandé pour avoir les noms en français).

__init__(speaker: Speaker)¶

Initialise le mode « Date et heure ».

Paramètres¶

speakerSpeaker

Instance du Speaker central utilisée pour prononcer les messages.

on_enter() → None¶

Hook appelé lorsque l'on ARRIVE sur ce mode.

Ici, on ne fait rien : - le ModeManager se charge déjà d'annoncer le nom du mode, - cela évite d'ajouter du bruit vocal à chaque changement de mode.

Ce hook existe principalement pour symétrie et pour d'éventuelles évolutions (par ex. « rafraîchir » un cache, etc.).

on_exit() → None¶

Hook appelé lorsque l'on QUITTE ce mode.

Aucun nettoyage spécifique n'est nécessaire pour ce mode, donc l'implémentation est vide pour le moment.

on_short_press() → None¶

Appui court : annonce l'heure système actuelle.

Exemple de rendu vocal :

« Il est 14 heures 32 »

Implémentation¶

- Récupère l'heure courante via `datetime.now()`.
- Formate l'heure avec `strftime(« %H heures %M »)`.
- Envoie la phrase complète au Speaker.

on_long_press() → None¶

Appui long : annonce la date du jour.

Exemple de rendu vocal :

« Nous sommes le mardi 5 juin 2025 »

Implémentation¶

- Utilise `datetime.now()` pour récupérer la date actuelle.
- **Décompose :**
 - jour de la semaine : `%A`
 - jour du mois : `%d` (converti en int pour éviter les zéros en tête)
 - mois : `%B`
 - année : `%Y`
- Construit une phrase en français et la passe au Speaker.

Remarque¶

- Selon la configuration locale du système (locale), `%A` et `%B` peuvent être en anglais. Pour forcer le français, il est recommandé de configurer la locale en `fr_FR` au niveau OS.

`modes.mode_dummy module`¶

Mode de démonstration / test.

Ce mode n'a aucune logique métier réelle : il sert uniquement à valider les interactions entre :

- le sélecteur rotatif (rotations, appui court, long, double),
- le ModeManager,
- la synthèse vocale (Speaker).

Il est utile comme « mode de développement » pour vérifier que l'enchaînement complet hardware -> callbacks -> mode -> TTS fonctionne correctement.

Ce mode peut servir de modèle minimal pour créer rapidement un nouveau mode.

`class modes.mode_dummy.ModeDummy(speaker)`¶

Bases : Mode

Mode « test » basique.

Comportement : - `on_enter` -> ne fait rien - `on_short_press` -> annonce un message simple - `on_long_press` -> annonce un message spécifique à l'appui long - `on_double_press` -> confirme le détecteur de double-clic

Ce mode n'effectue aucune action matérielle.

`__init__(speaker)`

Paramètres:

speaker (*Speaker*) – Synthèse vocale centrale, utilisée pour les annonces.

`on_enter()`

Appelé quand on arrive sur ce mode. Aucun feedback vocal ici pour ne pas surcharger : le ModeManager annonce déjà « Mode X : Mode test ».

`on_exit()`

Appelé lorsqu'on quitte ce mode. Rien à nettoyer.

`on_short_press()`

Annonce une confirmation d'appui court.

`on_long_press()`

Annonce une confirmation d'appui long.

`on_double_press()`

Annonce un double appui détecté.

`modes.mode_multimeter module`

Mode « Multimètre OWON 16 » pour l'assistant.

Ce module encapsule toute la logique « métier » côté utilisateur pour le multimètre :

- **Gestion de la voix :**
 - Toutes les annonces passent par le Speaker central.
- **Gestion du matériel :**
 - Dialogue avec le multimètre via OwonMultimeterDriver (qui lui-même utilise le transport BLE générique BleClient).
- **Logique UX :**
 - Appui court : lire la dernière mesure.
 - Appui long : annoncer l'état de connexion du multimètre.
 - Double appui : activer / désactiver une lecture automatique périodique (toutes les X secondes).

Le mode ne contient AUCUN code BLE direct et ne gère pas les broches GPIO : il consomme simplement un driver de plus bas niveau.

```
class modes.mode_multimeter.ModeMultimetre(speaker: Speaker, *,  
auto_read_interval: float = 5.0, driver_config: Dict[str, Any] | None = None)¶
```

Bases : Mode

Mode « Multimètre » de l'assistant.

Rôle¶

Ce mode fournit une interface vocale par-dessus le multimètre OWON 16 : il traduit les actions utilisateur (appuis sur le bouton, double-clic, sélection du mode) en :

- commandes haut niveau vers OwonMultimeterDriver (start/stop),
- annonces vocales lisibles pour un utilisateur non voyant.

Comportement UX¶

- **on_enter :**
 - démarre la recherche / connexion BLE en arrière-plan via le driver.
 - remet à zéro le mode de lecture automatique (on revient en manuel).
 - l'annonce « Mode X : Multimètre » est gérée par le ModeManager.
- **on_exit :**
 - arrête proprement le driver OWON (scan + connexion BLE).
 - désactive la lecture automatique et annule les timers.
- **on_short_press (clic simple) :**
 - annonce la dernière mesure disponible (ex : « 3.27 VOLT DC »).
 - si aucune mesure n'est encore disponible : message d'erreur vocal.
- **on_long_press (appui long) :**
 - annonce l'état de connexion actuel (connecté / en recherche).
- **on_double_press (double clic) :**
 - active / désactive un mode de lecture automatique : toutes les self._auto_read_interval secondes, la dernière mesure est annoncée, tant que le multimètre est connecté.

Détails techniques¶

- La conversion « nombre -> texte » est centralisée dans _format_value_for_speech() afin de gérer proprement :
 - les valeurs négatives (« moins 1.3 »),
 - le format numérique passé au TTS.

- La lecture automatique est pilotée par un threading.Timer qui se réarme lui-même (_schedule_auto_read / _auto_read_tick).
- Quand la connexion BLE est perdue, le mode auto reste activé mais ne parle pas tant que le multimètre n'est pas reconnecté.

`__init__(speaker: Speaker, *, auto_read_interval: float = 5.0, driver_config: Dict[str, Any] | None = None)`

Initialise le mode Multimètre.

Paramètres

speaker**Speaker**

Instance du Speaker central utilisée pour toutes les annonces vocales.

on_enter() → **None**

Hook appelé quand on arrive sur ce mode via le sélecteur rotatif.

Actions réalisées

- Désactive la lecture automatique (on revient en mode manuel).
- Annule tout timer éventuellement encore actif.
- **Démarre le driver OWON :**
 - scan BLE en arrière-plan,
 - tentatives de connexion automatiques.

Remarque

Le ModeManager se charge déjà d'annoncer « Mode X : Multimètre », donc on ne parle pas ici pour éviter les doublons.

on_exit() → **None**

Hook appelé quand on quitte ce mode.

Actions réalisées

- Désactive le mode de lecture automatique.
- Annule le timer de lecture automatique.
- Arrête le driver OWON (arrêt des scans / connexions BLE).

on_short_press() → **None**

Appui court : annonce la dernière mesure reçue.

Comportement

- Si aucune mesure n'est disponible (pas encore de trame reçue) : annonce « Aucune mesure disponible pour le moment. ».

- Sinon : * récupère le dictionnaire simplifié via `get_last_measure()`, * formate la valeur pour la voix (`_format_value_for_speech`), * concatène la valeur + le nom d'unité tel que décodé par le driver

(ex : « MILLI VOLT DC », « OHM NORMAL », « CELSIUS »...),

- envoie la phrase au Speaker.

Exemple de rendu ¶

« moins 1.3 MILLI VOLT DC »

on_long_press() → None ¶

Appui long : annonce l'état de connexion actuel du multimètre.

Cas gérés ¶

- **Si `OwonMultimeterDriver.is_connected` est True :**
-> « Le multimètre est connecté. »
- **Sinon :**
-> « Recherche ou connexion au multimètre en cours.
Veuillez patienter ou vérifier qu'il est allumé. »

on_double_press() → None ¶

Double appui : bascule entre lecture automatique et lecture manuelle.

Comportement ¶

- **Si la lecture automatique est désactivée :**
 - l'active,
 - annonce « Lecture automatique du multimètre activée. »,
 - planifie le premier tick avec `_schedule_auto_read()`.
- **Si la lecture automatique est déjà active :**
 - la désactive,
 - annule le timer associé,
 - annonce « Lecture automatique désactivée. Retour au mode manuel. ».

modes.mode_reading_machine module ¶

Mode « Machine à lire ».

Ce module orchestre le flux utilisateur le plus complet du projet :

1. Détection de la disponibilité de la caméra.

2. Capture d'une image via PiCamera.
3. OCR et nettoyage du texte via TextReaderDevice.
4. Synthèse du texte en WAV via Speaker.synthesize_to_file.
5. Lecture longue contrôlable via Speaker.play.

Le mode ne lit pas directement le keypad GPIO. Il reçoit les touches non-globales depuis ModeManager, ce qui permet de conserver :

- les touches globales volume/vitesse dans tous les modes ;
- une logique de contrôle métier isolée ici ;
- une implémentation testable avec des faux objets.

**class modes.mode_reading_machine.ModeReadingMachine(speaker: *Speaker*, *,
reading_config: Dict[str, Any] | None = None, ocr_config: Dict[str, Any] | None = None,
camera_config: Dict[str, Any] | None = None)¶**

Bases : Mode

Mode « Machine à lire » : capture caméra -> OCR -> synthèse -> lecture.

Ce mode ne pilote pas directement le keypad matériel : il reçoit les touches non-globales via ModeManager.on_key_pressed().

Commandes par défaut¶

- 1 : capturer et lancer le pipeline complet.
- 3 : annuler la lecture ou le pipeline.
- 4 : reculer dans l'audio.
- 5 : pause / reprise.
- 6 : avancer dans l'audio.
- * : relire le dernier WAV généré.

Contraintes importantes¶

- Le pipeline est lancé dans un thread dédié pour ne pas bloquer l'interface globale.
- Les messages d'erreur et de statut sont joués sous forme de sons courts afin d'éviter une concurrence trop forte avec la lecture longue du document.

KEY_CAPTURE = '1'
KEY_CANCEL = '3'
KEY_BACKWARD = '4'
KEY_PLAY_PAUSE = '5'
KEY_FORWARD = '6'
KEY_REPLAY = '*'
on_enter() → None

Prépare le mode à l'utilisation.

Actions réalisées : - réinitialise l'état d'annulation ; - sonde la disponibilité de la caméra ; - lance le thread de surveillance si la caméra est absente.

on_exit() → None

Nettoie tout ce qui a été démarré par le mode.

Cette méthode doit laisser le mode dans un état neutre pour un éventuel retour ultérieur, sans fuite de lecture ou de thread.

on_short_press() → None

Aucun comportement sur le bouton rotatif dans ce mode.

La machine à lire utilise le keypad pour éviter de surcharger le bouton unique du sélecteur avec trop de commandes.

on_long_press() → None

Aucun comportement sur l'appui long du bouton rotatif.

Les commandes sont volontairement regroupées sur le clavier 4x4.

on_keypad_key(key: str) → None

Route une touche du keypad vers la commande métier correspondante.

Seules les touches non-globales arrivent ici : volume et vitesse sont interceptés plus haut par ModeManager.

modes.mode_thermometer module

modes/mode_thermometer.py

Mode « Thermomètre WiFi » pour l'assistant.

Rôle :

- Interface entre l'utilisateur (boutons, sélecteur) et le Driver Thermomètre.
- Ce mode suppose que la Pi et l'ESP32 sont sur le même réseau WiFi existant.

Comportement UX :¶

- on_enter : Démarrage du driver (recherche du capteur sur le réseau).
- on_exit : Arrêt du driver.
- Appui court : Lit la température (« Il fait 22 virgule 5 degrés »).
- Appui long : Annonce l'état (« Le thermomètre est connecté au réseau local »).

class modes.mode_thermometer.ModeThermometre(speaker: *Speaker*)¶

Bases : Mode

Mode Thermomètre connecté via le réseau local.

Le mode s'appuie sur ThermometerDriver pour toute la partie communication. Son rôle est volontairement limité à la logique UX : formater la valeur pour la voix et répondre aux appuis utilisateur.

__init__(speaker: *Speaker*)¶

Initialise le mode avec sa synthèse vocale et son driver.

on_enter() → None¶

Arrivée sur le mode Thermomètre : On lance le driver qui va chercher l'ESP32 sur le réseau local.

on_exit() → None¶

Sortie du mode : On arrête le driver pour économiser des ressources réseau.

on_short_press() → None¶

Appui court : Annonce la température.

on_long_press() → None¶

Appui long : Annonce l'état de connexion.

Module contents¶